

Mod-4 congruence and parity laws for the residue-class recurrences of chorded-cycle matching counts (Lean 4 machine-checked claim of record)

Aurora0134*

September 28, 2026

Abstract

Let $a(n)$ be the number of matchings (including the empty one) of the graph on $\mathbb{Z}/n\mathbb{Z}$ with the cycle edges plus all chords $\{i, i + \lfloor n/3 \rfloor\}$. Exact enumeration for $n \geq 3$ starts 4, 7, 11, 51, 99, 191, 382, 780, $\dots$; the sequence splits by $n \bmod 3$ into three subfamilies whose terms are reproduced, up to $n = 150$, by integer recurrences of orders 6, 8 and 8 fitted over $\mathbb{Q}$ with Berlekamp–Massey. We state and machine-check, in Lean 4 (v4.34.0, mathlib v4.34.0), three congruence laws for those recurrence-defined sequences: constant $3 \bmod 4$; a two-valued mod4 law along $j+1 \bmod 3$ for $j \geq 1$; and parity characterised by $(j+1) \bmod 5 = 3$. Every proof compiles with no `sorry` and `#print axioms` reports only `{propext, Quot.sound, Classical.choice}`. Novelty adjudication: no occupying record found in OEIS, zbMATH, Math Stack Exchange, arXiv or the public Lean ecosystem under the query discipline recorded in the deposit; value tier *new sequence / new recurrence*, not new mathematics. The combinatorial identification is not kernel-proved and is stated as a conjecture. This note is a priority claim of record, not a full paper.

1 Introduction

Three theorems about three integer sequences defined by initial values and linear recurrences are reported here; the sequences come from counting

*Contact: via the GitHub account Aurora0134; ORCID to be added at Zenodo upload.

matchings of cycles augmented by chords of length $\lfloor n/3 \rfloor$, and the recurrences were obtained by exact rational fitting of enumerated data [5, 4]. The proofs were produced by the AI4Math pipeline, in which LLM workers generate statements and proof scripts and the Lean 4 kernel is the sole adjudicator at every gate: statements are frozen by hash before proving, and a separate audit re-computes the symmetric difference, scans the whole file for `sorry/admit` including comments, and re-prints the axiom dependencies [1, 2]. Verification strength is stated below verbatim. This note is a priority claim of record deposited so that a dated, citable public record exists ahead of any narrative paper; the fuller paper for the same three theorems is prepared separately and will reference this deposit. No literature review is attempted here.

2 Statement

Throughout, the three sequences are *defined* by the displayed initial values and recurrences; 0-based indexing, with $c_r(j)$ corresponding to the combinatorial $a(3(j+1)+r)$.

Theorem 1 (constant congruence). *For the sequence c_2 of Listing 1 (nine seeds, order-8 recurrence), $c_2(k) \equiv 3 \pmod{4}$ for every $k \in \mathbb{N}$.*

Listing 1: Frozen formal statement, verbatim lines 22–42 of `formalized/01-cchord-mods.lean` (sha256 a69d00e5).

```

1  /-- **序列 cchordR2 (r=2 子族, 阶 8 递推) **.
2  初值 c(0..8) = 11, 191, 1475, 10615, 77623, 565895, 4127167,
      30100411, 219522983
3  (= counts.txt 中 a(5), a(8), a(11), a(14), a(17), a(20), a(23), a(26), a
      (29)) ;
4  递推 c(n+9) = 4c(n+8)+22c(n+7)+18c(n+6)-22c(n+5)-16c(n+4)+10c(n
      +3)+2c(n+2)-c(n+1)。 -/
5  def cchordR2 : ℕ → ℤ
6    | 0 => 11
7    | 1 => 191
8    | 2 => 1475
9    | 3 => 10615
10   | 4 => 77623
11   | 5 => 565895
12   | 6 => 4127167
13   | 7 => 30100411
14   | 8 => 219522983

```

```

15 | n + 9 =>
16   4 * cchordR2 (n + 8) + 22 * cchordR2 (n + 7) + 18 * cchordR2
    (n + 6)
17   - 22 * cchordR2 (n + 5) - 16 * cchordR2 (n + 4) + 10 *
    cchordR2 (n + 3)
18   + 2 * cchordR2 (n + 2) - cchordR2 (n + 1)
19
20 /-- **C1 (主攻) **:  $r=2$  子族  $\bmod 4$  恒等于 3。 -/
21 theorem cchordR2_mod_four :  $\forall k : \mathbb{N}, \text{cchordR2 } k \equiv 3 \text{ [ZMOD 4]}$  :=
    by

```

Theorem 2 (two-valued congruence along $j + 1 \bmod 3$). *For the sequence c_0 of Listing 2 (seven seeds, order-6 recurrence) and every $j \geq 1$, $c_0(j) \equiv 2 \pmod{4}$ if $3 \mid (j + 1)$ and $c_0(j) \equiv 3 \pmod{4}$ otherwise. The hypothesis is needed: $c_0(0) = 4 \equiv 0 \pmod{4}$.*

Listing 2: Frozen formal statement, verbatim lines 45–63 of formalized/01-cchord-mods.lean.

```

1 /-- **序列 cchordR0 ( $r=0$  子族, 阶 6 递推) **.
2 初值  $c(0..6) = 4, 51, 382, 2743, 19907, 144054, 1043147$ 
3 ( $= \text{counts.txt}$  中  $a(3), a(6), a(9), a(12), a(15), a(18), a(21)$ ) ;
4 递推  $c(n+7) = 5c(n+6) + 16c(n+5) + 3c(n+4) - 10c(n+3) - c(n+2) + c(n+1)$ .
  -/
5 def cchordR0 :  $\mathbb{N} \rightarrow \mathbb{Z}$ 
6 | 0 => 4
7 | 1 => 51
8 | 2 => 382
9 | 3 => 2743
10 | 4 => 19907
11 | 5 => 144054
12 | 6 => 1043147
13 | n + 7 =>
14   5 * cchordR0 (n + 6) + 16 * cchordR0 (n + 5) + 3 * cchordR0
    (n + 4)
15   - 10 * cchordR0 (n + 3) - cchordR0 (n + 2) + cchordR0 (n +
    1)
16
17 /-- **C2 (伴随) **:  $r=0$  子族  $\bmod 4$  三分支 ( $j \geq 1$  起;  $j=0$  处  $\equiv 0$  为已知
    首例外)。 -/
18 theorem cchordR0_mod_four (j :  $\mathbb{N}$ ) (hj :  $1 \leq j$ ) :
19   cchordR0 j  $\equiv$  (if  $3 \mid (j + 1)$  then 2 else 3) [ZMOD 4] := by

```

Theorem 3 (parity characterisation). *For the sequence c_1 of Listing 3 (nine seeds, order-8 recurrence) and every $j \in \mathbb{N}$, $2 \mid c_1(j)$ if and only if $(j + 1) \bmod 5 = 3$.*

Listing 3: Frozen formal statement, verbatim lines 66–86 of `formalized/01-cchord-mods.lean`.

```

1  /-- **序列 cchordR1 (r=1 子族, 阶 8 递推) **.
2  初值 c(0..8) = 7, 99, 780, 5591, 41281, 302139, 2217121, 16257026,
    119231297
3  (= counts.txt 中 a(4), a(7), a(10), a(13), a(16), a(19), a(22), a(25), a
    (28)) ;
4  递推 c(n+9) = 4c(n+8)+23c(n+7)+14c(n+6)-23c(n+5)-14c(n+4)+9c(n+3)
    +2c(n+2)-c(n+1)。 -/
5  def cchordR1 : ℕ → ℤ
6  | 0 => 7
7  | 1 => 99
8  | 2 => 780
9  | 3 => 5591
10 | 4 => 41281
11 | 5 => 302139
12 | 6 => 2217121
13 | 7 => 16257026
14 | 8 => 119231297
15 | n + 9 =>
16   4 * cchordR1 (n + 8) + 23 * cchordR1 (n + 7) + 14 * cchordR1
    (n + 6)
17   - 23 * cchordR1 (n + 5) - 14 * cchordR1 (n + 4) + 9 *
    cchordR1 (n + 3)
18   + 2 * cchordR1 (n + 2) - cchordR1 (n + 1)
19
20 /-- **C3 (伴随) **: r=1 子族奇偶 iff——偶数当且仅当 (j+1) % 5 = 3。
    -/
21 theorem cchordR1_even_iff (j : ℕ) : 2 ∣ cchordR1 j ↔ (j + 1) % 5
    = 3 := by

```

Conjecture 1 (combinatorial bridge, not proved here). For $r \in \{0, 1, 2\}$ and all $j \geq 0$, $c_r(j) = a(3(j + 1) + r)$, where a counts matchings of the chorded cycle described in the abstract.

3 Proof

All three proofs are strong induction on the index with the seed window discharged by computation and the step closed by congruence algebra inside `Int.ModEq`; Theorems 2 and 3 add a case split on $(j + 1) \bmod 3$ and $(j + 1) \bmod 5$ respectively. The complete proof of Theorem 1 is reproduced verbatim below. The other two declarations are longer — Theorem 2 spans lines 86–137 and Theorem 3 lines 159–262 of the same final file — and are excerpted here only where the argument has a distinctive shape: the closing assembly of Theorem 3 from its modulo-2 strengthening, which is where the if-and-only-if is won. The complete 262-line file ships in this deposit under `proofs/`.

Listing 4: Complete proof of Theorem 1, verbatim lines 43–67 of `final/01-cchord-proved.lean` (sha256 af39d35e).

```
1 theorem cchordR2_mod_four :  $\forall k : \mathbb{N}, \text{cchordR2 } k \equiv 3 \text{ [ZMOD 4]}$  :=  
2   by  
3   intro n  
4   induction n using Nat.strong_induction_on with  
5   | _ n ih =>  
6     rcases lt_or_ge n 9 with h | h  
7     · interval_cases n <=> decide  
8     · obtain ⟨j, rfl⟩ :  $\exists j, n = j + 9$  := ⟨n - 9, by omega⟩  
9       have rec9 :  $\forall m : \mathbb{N}, \text{cchordR2 } (m + 9)$   
10         = 4 * cchordR2 (m + 8) + 22 * cchordR2 (m + 7) + 18 *  
11         cchordR2 (m + 6)  
12         - 22 * cchordR2 (m + 5) - 16 * cchordR2 (m + 4) + 10 *  
13         cchordR2 (m + 3)  
14         + 2 * cchordR2 (m + 2) - cchordR2 (m + 1) := fun m =>  
15     by  
16       simp only [cchordR2]  
17       rw [rec9 j]  
18       have h8 : cchordR2 (j + 8)  $\equiv 3 \text{ [ZMOD 4]}$  := ih (j + 8) (by  
19         omega)  
20       have h7 : cchordR2 (j + 7)  $\equiv 3 \text{ [ZMOD 4]}$  := ih (j + 7) (by  
21         omega)  
22       have h6 : cchordR2 (j + 6)  $\equiv 3 \text{ [ZMOD 4]}$  := ih (j + 6) (by  
23         omega)  
24       have h5 : cchordR2 (j + 5)  $\equiv 3 \text{ [ZMOD 4]}$  := ih (j + 5) (by  
25         omega)  
26       have h4 : cchordR2 (j + 4)  $\equiv 3 \text{ [ZMOD 4]}$  := ih (j + 4) (by  
27         omega)  
28       have h3 : cchordR2 (j + 3)  $\equiv 3 \text{ [ZMOD 4]}$  := ih (j + 3) (by
```

```

20   omega)
    have h2 : cchordR2 (j + 2) ≡ 3 [ZMOD 4] := ih (j + 2) (by
21   omega)
    have h1 : cchordR2 (j + 1) ≡ 3 [ZMOD 4] := ih (j + 1) (by
22   omega)
    have key := ((((((h8.mul_left 4).add (h7.mul_left 22)).add
(h6.mul_left 18)).sub
23   (h5.mul_left 22)).sub (h4.mul_left 16)).add (h3.mul_left
10)).add
24   (h2.mul_left 2)).sub h1
25   exact key.trans (by decide)

```

Listing 5: Excerpt: assembly of Theorem 3 from the modulo-2 strengthening main proved at lines 160–251 of the same file; verbatim lines 252–262 (contiguous block; the complete proofs of Theorem 2 and Theorem 3 are lines 86–137 and 159–262, in proofs/).

```

1   have h := main j
2   rw [← Int.modEq_zero_iff_dvd]
3   constructor
4   · intro hd
5     by_cases hc : (j + 1) % 5 = 3
6     · exact hc
7     · rw [if_neg hc] at h
8       exact absurd (h.symm.trans hd) (by decide)
9   · intro hc
10    rw [if_pos hc] at h
11    exact h

```

4 Verification evidence

- Toolchain: Lean v4.34.0 (leanprover/lean4:v4.34.0), mathlib4 v4.34.0, revision 5ed29652. Reproduction: with that pin, lake env lean on proofs/01-cchord-proved.lean compiles the file with exit code 0 (measured 29.1 s for the strict pass).
- Statement integrity: the frozen statement file formalized/01-cchord-mods.lean (87 lines) has sha256 a69d00e537675c40bd13e82344004e7f1abd370dc3d9e543396fbf1818c3a03b; the final file final/01-cchord-proved.lean (262 lines) has sha256 af39d35ea29c9a01369473140319f9c330d326601463839875d11b14bbba2237. The final file was compared with the frozen file by symmetric difference over the statement region:

zero drift; the only differences are the three proof bodies replacing the `sorry` placeholders and one header-comment line.

- Kernel check: compiles with 0 `sorry` (whole-file scan including comments, case-insensitive, also for `admit/sorryAx`); `#print axioms` output, verbatim (kept literal; the smaller monospace width is typesetting only):

```
1 'cchordR2_mod_four' depends on axioms: [propext, Classical.choice,
    Quot.sound]
2 'cchordR0_mod_four' depends on axioms: [propext, Classical.choice,
    Quot.sound]
3 'cchordR1_even_iff' depends on axioms: [propext, Classical.choice,
    Quot.sound]
```

The strict re-compile emits 68 deprecation warnings (`if_neg/if_pos`) and no error; warnings are not treated as a refusal ground by the audit, and are recorded here. The log ships as `audit/audit-strict-compile.log`, with machine-specific path prefixes replaced by `<repo>/` for the public deposit.

- Grading by the audit department: all three theorems *complete proof*; lowest grade reached for the case = *complete proof*; no scaffolding route was used.
- Audit chain (originals in this deposit under `audit/`): final adjudication `final-01-cchord.txt` (four independent checks), well-definedness verdict `welldef-verdict.md` with gate logs `gate-c1.txt`, `gate-c2.txt`, `gate-c3.txt` and the rechecker log `gate-recheck-frozen.txt`, axiom printout `audit-axioms-sidecar.log` (the three lines quoted above are its whole content), and this note's claim check `claim-check.md` — added to `audit/` after the audit department returns its adjudication, and a prerequisite for publication. The semantic-fidelity check (roundtrip re-translation) of the underlying task ran on the same model node as its formalisation; that independence weakness is mitigated, not removed, by kernel adjudication and manual review.

5 Novelty evidence

Adjudication copied from the pipeline's exclusivity review (C9 gate), which ran before the problem was accepted and re-derived the data independently:

verdict **no occupying record found**, value tier **new sequence / new recurrence**. It is not a claim of new mathematics and not a first-discovery claim. Summary by channel; the full query log, including every zero-hit query, is in this deposit at `audit/c9-record.md`.

channel	queries	result
OERS sequence search [7]	3	zero hits: supplied seed and its drop-first / drop-two variants (3), real sequence and its drop-first / drop-two variants (3), $\times 2$ (1), $\times 1$ (1), tail segment (1)
OERS per-residue subsequences	3	zero hits (4, 51, 382, 2743, 7, 99, 780, 5591, 11, 191, 1476, 10615)
OERS perfect-matching variant	2	zero hits
OERS keywords	2	10 hits each, none matching counts of this family
zbMATH	4	Boseya index circulant graph : clean zero reading; three other queries return hits in other directions (Pfaffian property; pancyclicity and Hamilton-type chorded-cycle results [6]; matching polynomials)
Math Stack Exchange, arXiv	2	zero hits
OpenAlex	1	HTTP 429 on this candidate, not verified (gap disclosed)
public Lean ecosystem	2	zero hits; one repository tree walk rate-limited
library layer	3	mathlib has <code>circulantGraph</code> [3] but no matching-count theorem; compiles none; sequence(s) none

Two points of the record are part of the evidence. The externally supplied seed began $2, 2, \dots$, and those two terms are artefacts of that pipeline’s degenerate $n = 1, 2$ cases; the sequence used here starts at $n = 3$ with 4. And the general web search layer was unavailable during the review, so literature coverage is zbMATH-, arXiv- and Math-Stack-Exchange-centric; a Chinese-language database was not searched. If any of these channels later returns a hit on chorded-cycle matching counts, the tier above weakens accordingly, and this deposit will be amended by a later version rather than by editing this record.

6 Scope and limitations

Grade and boundary, stated as the audit states them. (a) The theorems cover the recurrence-defined sequences only; their equality with matching counts of the chorded cycle is not kernel-proved and is Conjecture 1. Its empirical support is enumeration by exact programs: the counting main program produced $3 \leq n \leq 150$, cross-checked against a brute-force enumerator for $3 \leq n \leq 16$ and against a second layout for $3 \leq n \leq 36$, and against the externally supplied seed only for $3 \leq n \leq 15$; the enumerators themselves are not verified, and the scripts and their outputs stay in the working repository under `tasks/20260927-mossad-cchord/`. (b) Theorem 2 holds for $j \geq 1$; at $j = 0$ the value is 4, which is $\equiv 0 \pmod{4}$. (c) Indexing is 0-based: $c_r(j)$ corresponds to $a(3(j+1)+r)$. (d) The recurrences are computational findings: Berlekamp–Massey fits over $\mathbb{Q}$ verified digit-for-digit against enumeration, not derived from a transfer-matrix argument, and the natural uniform step-3 ansatz is refuted on the enumerated window by exact rational arithmetic (136 failures out of 139 test indices, $12 \leq n \leq 150$) — a computation, not a theorem of this note. (e) The gate-four sign-off of the underlying task was exercised by the author’s instruction to deposit this claim on 2026-09-27, while the task report file still carries its draft header;

both facts are recorded here as they stand. (f) This note carries no literature review and is a deposit record, not a survey; the narrative paper for the same result is separate.

Data availability

The deposit package for this claim contains the claim note (LaTeX source and PDF), the Lean sources (`proofs/`: frozen statement file, final proof file, `lean-toolchain`), the verification and audit logs, and the full C9 query log with zero-hit entries (`audit/`). Zenodo: the DOI is reserved at upload by the author and is recorded in `metadata/README.md` of this package once created; see `UPLOAD.md` in the source repository. Public mirror of this deposit: the claim lane’s repository <https://github.com/Aurora0134/ai4math-claims>, directory `chorded-cycle-mod4/` (pushed under the author’s standing instruction; the commit and timestamp are recorded in the source repository’s `claims/chorded-cycle-mod4/card.md`). Claim note under CC BY 4.0, code under MIT.

Use of generative AI

(a) AI performed: candidate generation and the exclusivity review, autoformalisation of the three statements, the roundtrip semantic check, proof search and proof-script writing, error classification and repair, and the assembly of this note including its verbatim listings. (b) Model, node and budget: all task-stage sampling ran on one node, wire-id `anthropic/a6api-main/kimi-k3`, with no cross-node switching, and consumed 1 LLM call of a permitted 8 and 15 kernel compiles of a permitted 24, copied verbatim from the task ledger. This note consumed 0 pipeline LLM calls against a cap of 4; the drafting session ran with its model selection key in auto-routing mode (last observed relay node `stepfun/step-5-preview`), which is disclosed rather than replaced by the task node’s name. (c) Final adjudication: all statements and proofs reported here were checked by the Lean 4 kernel: every proof compiles with no `sorry`, and `#print axioms` reports only `{propext, Quot.sound, Classical.choice}`. (d) The AI system is not an author; the human author reviewed the evidence and is responsible for all content.

Author contributions

CRediT-style: the human author adjudicated topic selection, statement semantics and the deposit decision; the AI4Math pipeline (an LLM) generated the candidate propositions, the proof searches and the text drafts. The AI system is not listed as an author.

References

- [1] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *Automated Deduction – CADE-28*, LNCS, pp. 625–635, 2021. https://doi.org/10.1007/978-3-030-79876-5_37
- [2] The mathlib Community. The Lean mathematical library. *CPP 2020*, pp. 367–381, 2020. <https://doi.org/10.1145/3372885.3373824>
- [3] I. Renison and B. Mehta. `Mathlib/Combinatorics/SimpleGraph/Circulant.lean`, `mathlib4`, tag `v4.34.0`. <https://raw.githubusercontent.com/leanprover-community/mathlib4/v4.34.0/Mathlib/Combinatorics/SimpleGraph/Circulant.lean>
- [4] C. D. Godsil and I. Gutman. On the theory of the matching polynomial. *Journal of Graph Theory* 5 (1981), pp. 137–144. <https://doi.org/10.1002/jgt.3190050203>
- [5] J. L. Massey. Shift-register synthesis and BCH decoding. *IEEE Transactions on Information Theory* 15 (1969), pp. 122–127. <https://doi.org/10.1109/TIT.1969.1054260>
- [6] B. Elliott, R. Gould and K. Hirohata. On degree sum conditions and vertex-disjoint chorded cycles. *arXiv:1911.08686 [math.CO]*, 2019. <https://arxiv.org/abs/1911.08686>
- [7] The OEIS Foundation. *The On-Line Encyclopedia of Integer Sequences*. <https://oeis.org/>